CMS User Guide

calildrissi.dev — Content Management System

1. Set Up Environment Variables

Go to **Netlify Dashboard → Site settings → Environment variables** and add:

Variable	Value
`GITHUB_REPO`	`CalilDrissi/calil-drissi-portfolio`
`GITHUB_TOKEN`	A GitHub Personal Access Token with `repo` scope Generate at github.com/settings/tokens
`ADMIN_PASSWORD`	Any password you choose for admin login

> After adding variables, trigger a redeploy: **Deploys → Trigger deploy → Deploy site**.

2. Access the Admin Panel

Navigate to `https://calil-drissi.netlify.app/admin/` and log in with your `ADMIN_PASSWORD`.

3. Edit Portfolio Content

1. Select **Portfolio** in the sidebar
2. Switch between **EN** and **FR** tabs to edit each language
3. Edit any section: intro, projects, awards, about, recognition, brands, connect, statement, etc.
4. Click **Save** — this commits the JSON file directly to your GitHub repo
5. Netlify auto-rebuilds and redeploys the site within ~30 seconds

4. Manage Blog Posts

1. Select **Blog Posts** in the sidebar
2. Switch between **EN** / **FR** tabs

Create a Post

Click **New Post**, fill in title, date, excerpt, tags, cover image, and write the body using the rich text editor (Quill). The slug is auto-generated from the title.

Edit a Post

Click on any existing post from the list to open it in the editor.

Delete a Post

Click the delete button on any post. A confirmation dialog will appear. Only blog files can be deleted.

5. How the Build Works

```
content/en.json + content/fr.json → Portfolio pages (/ and /fr/)
content/blog/en/*.json → Blog posts at /blog/{slug}/
content/blog/fr/*.json → Blog posts at /fr/blog/{slug}/
index.html → Template (don't edit in admin)
build.js → Runs on deploy, generates static HTML
```

Every save from the admin panel commits to GitHub → triggers Netlify build → `node build.js` runs → static HTML is generated and deployed.

6. Adding Content Locally

You can also edit the JSON files directly in your code editor:

```
# Edit portfolio content
vim content/en.json

# Add a blog post
cat > content/blog/en/my-post.json << 'EOF'
{
  "slug": "my-post",
  "title": "My Post Title",
  "date": "2026-03-09",
```

```
  "excerpt": "A short description...",
  "tags": ["JavaScript", "React"],
  "coverImage": "https://images.unsplash.com/...",
  "body": "<p>Your HTML content here.</p>"
}
EOF

# Build and preview locally
node build.js
open dist/index.html

# Deploy
git add . && git commit -m "Update content" && git push
```

7. Site URLs

Page	URL
Portfolio (EN)	`https://calil-drissi.netlify.app`
Portfolio (FR)	`https://calil-drissi.netlify.app/fr/`
Blog (EN)	`https://calil-drissi.netlify.app/blog/`
Blog (FR)	`https://calil-drissi.netlify.app/fr/blog/`
Admin Panel	`https://calil-drissi.netlify.app/admin/`

8. Blog Post JSON Format

Field	Type	Description
`slug`	string	URL-friendly identifier (auto-generated from title)
`title`	string	Post title
`date`	string	Publication date (YYYY-MM-DD)
`excerpt`	string	Short description for listing cards
`tags`	array	Category tags, e.g. ["React", "CSS"]
`coverImage`	string	URL for the cover image

`body`	string	HTML content of the post

9. WordPress Local Setup (Alternative Blog Workflow)

You can use a local WordPress instance as a writing environment, then export posts as static JSON for the build.

9.1 Prerequisites

- Docker Desktop installed and running
- Node.js 18+ (for native `fetch`)

9.2 Create docker-compose.yml

Create this file at the project root (`/v2/docker-compose.yml`):

```
version: '3.8'
services:
  db:
    image: mysql:8.0
    volumes:
      - wp_db_data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8080:80"
    volumes:
      - wp_data:/var/www/html
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress

volumes:
  wp_db_data:
  wp_data:
```

9.3 Start WordPress

```
docker compose up -d
```

Visit `http://localhost:8080` to complete the WordPress setup wizard. Choose English as the language.

9.4 Install Polylang (Multilingual)

1. Go to `http://localhost:8080/wp-admin`
2. **Plugins** → **Add New** → Search "Polylang" → **Install** → **Activate**
3. **Languages** → Add **English** (en) and **French** (fr)
4. Set English as the default language

> With Polylang active, the REST API supports `?lang=en` and `?lang=fr` to filter posts by language.

9.5 Writing Blog Posts

1. Go to **Posts** → **Add New**
2. Write your post with title, content, tags, and a featured image
3. Set the language to **English** in the Polylang box
4. Click **Publish**
5. To add a French translation, click the **+** next to the FR flag in the Polylang box — this creates a linked translation

9.6 Export Posts to Static JSON

Run the sync script to fetch posts from WordPress and save them as JSON files:

```
# Fetch all posts from local WordPress and write to content/blog/
node sync-wp.js

# This creates/updates:
#   content/blog/en/{slug}.json
#   content/blog/fr/{slug}.json
```

The sync script calls the WordPress REST API:

```
GET http://localhost:8080/wp-json/wp/v2/posts?lang=en&per_page=100&_embed
```

```
GET http://localhost:8080/wp-json/wp/v2/posts?lang=fr&per_page=100&_embed
```

Each post is converted to the standard JSON format:

```
{
  "slug": "my-post-title",
  "title": "My Post Title",
  "date": "2026-03-15",
  "excerpt": "A short description...",
  "tags": ["React", "Performance"],
  "coverImage": "/blog/images/my-cover.jpg",
  "body": "<p>Your HTML content...</p>"
}
```

> **Images:** The sync script downloads images from WordPress (`wp-content/uploads/`) into `dist/blog/images/` and rewrites URLs so they work in production. You can also use external image URLs (Unsplash, Cloudinary) directly in WordPress to skip this step.

9.7 Build and Deploy

```
# Build the static site (reads the JSON files as usual)
node build.js

# Preview locally
npx http-server dist -p 3000

# Commit the synced posts and deploy
git add content/blog/
git commit -m "Sync blog posts from WordPress"
git push    # Netlify auto-builds
```

9.8 Full Workflow Summary

```
docker compose up -d → Start local WordPress
Write posts at localhost:8080/wp-admin → Use Polylang for EN/FR
node sync-wp.js → Fetch from WP API → save JSON files
node build.js → Generate static HTML from JSON
git push → Netlify deploys automatically
```

9.9 Stopping WordPress

```
# Stop containers (data persists)
docker compose stop

# Remove containers and volumes (deletes all data)
docker compose down -v
```

> **Netlify builds:** WordPress does not need to be running for Netlify deploys. The build reads from the committed JSON files in `content/blog/`. Only run WordPress locally when writing new posts.